

```
#include <thread>
void Foo(int a, int b) {...}
// the thread will execute foo with arguments 1, 2
std::thread t(Foo, 1, 2);
// [x, &y]: captured by value and reference
std::thread t([x, &y]() -> void { y=x*42; });
t.join(); // remember to join the thread!
```

Passing a reference to std::thread:

```
#include <functional> // std::ref
void ThreadBody(std::string by_value, std::string& by_reference) {...}
std::string hello = "Hello";
std::string world = "World";
// use std::ref to pass thread argument by reference
// use std::cref for const references
std::thread t(ThreadBody, hello, std::ref(world));
```

Vector of threads:

```
std::vector<std::thread> threads;
for (size_t i = 0; i < 16; i++) { // launch a bunch of threads
    threads.push_back(std::thread(ThreadBody, a, b, c));
    // alternative: construct an object in-place at end of vector
    // type inference determines that std::thread constructor should be
    // invoked with ThreadBody, a, b, c
    threads.emplace_back(ThreadBody, a, b, c); }
// wait for them to finish
for (auto& thread : threads){ thread.join(); }
```